

Чек-лист для тестирования клиентской части

Тестирование структуры проекта и сборки

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
1.1	Проверить наличие папок <code>network/</code> , <code>data/</code> , <code>ui/</code>	Все папки присутствуют в проекте	Визуальная проверка / <code>dir</code> (Windows) / <code>ls</code> (Linux)
1.2	Проверить наличие файла <code>CMakeLists.txt</code>	Файл существует, содержит настройки для Qt5/Qt6	Визуальная проверка / <code>find</code>
1.3	Собрать проект с CMake	Сборка проходит без ошибок	<code>cmake --build build</code> / Visual Studio
1.4	Проверить подключение зависимостей	Qt (Widgets, Network, Charts), QJsonValue, QJsonObject найдены	<code>cmake</code> вывод / <code>find_package</code>
1.5	Собрать проект с флагом <code>BUILD_QT=ON</code>	Собирается Qt-версия клиента	<code>cmake -DBUILD_QT=ON</code>
1.6	Собрать проект с флагом <code>BUILD_QT=OFF</code>	Собирается консольная версия клиента	<code>cmake -DBUILD_QT=OFF</code>

Тестирование HmiClient (Сетевая часть)

2.1. Подключение к серверу

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
2.1.1	Подключиться к серверу по валидному IP:port	Соединение установлено успешно	Встроенный клиент / Wireshark для проверки трафика
2.1.2	Подключиться к серверу по невалидному IP	Ошибка подключения, сообщение пользователю	Ручное тестирование / Mock-сервер
2.1.3	Подключиться к серверу по невалидному порту	Ошибка подключения, сообщение пользователю	Ручное тестирование / Mock-сервер
2.1.4	Разорвать соединение вызовом <code>disconnectFromHost()</code>	Соединение разорвано, статус обновлен	Юнит-тесты (JUnit / Google Test)
2.1.5	Переподключение после разрыва	Клиент пытается переподключиться автоматически	Интеграционные тесты / Симуляция обрыва сети
2.1.6	Таймаут подключения	Соединение не зависает, ошибка через заданное время	JUnit / Mock-сервер с задержкой

2.2. Отправка запросов

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
2.2.1	Отправить запрос <code>get_server_info</code>	Запрос сформирован корректно, отправлен	Wireshark / Mock-сервер / Логи
2.2.2	Отправить запрос с Transaction ID	Transaction ID инкрементируется для каждого запроса	Юнит-тесты / Логи
2.2.3	Отправить запрос с параметрами	JSON содержит поля <code>cmd</code> и <code>params</code>	QTest / Google Test
2.2.4	Формат кадра (4 байта + JSON)	Кадр соответствует спецификации (Little-Endian)	Wireshark / Mock-сервер
2.2.5	Отправить несколько запросов подряд	Все запросы отправлены, ID уникальны	Интеграционные тесты / Jmeter
2.2.6	Отправить запрос при разорванном соединении	Ошибка, сообщение пользователю	Ручное тестирование / Симуляция

2.3. Прием ответов

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
2.3.1	Получить корректный JSON-ответ	JSON парсится без ошибок	QTest / Google Test
2.3.2	Получить ответ с Transaction ID	ID совпадает с ID запроса	Юнит-тесты
2.3.3	Получить большой JSON-ответ	Полный ответ прочитан корректно	Wireshark / Mock-сервер
2.3.4	Получить ответ с ошибкой	Сообщение об ошибке отображается	Mock-сервер / QTest
2.3.5	Частичное чтение данных (TCP фрагментация)	Данные собираются корректно из нескольких пакетов	Симуляция TCP-фрагментации / Wireshark

Тестирование протокола "Сервер-НМІ"

3.1. Команда get_server_info

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
3.1.1	Отправить get_server_info	Получен ответ с connected, connected_time, endpoint, launch_time	Mock-сервер / Postman / Wireshark
3.1.2	Проверить статус connected	Значение true или false корректно отображается	QTest / Ручная проверка
3.1.3	Проверить время запуска сервера	Время отображается в формате YYYY.MM.DD HH:MM:SS.mmm	QTest / Регулярные выражения

3.2. Команда get_tanks_info

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
3.2.1	Отправить get_tanks_info	Получен список резервуаров с name, type, volume_max	Mock-сервер / Postman
3.2.2	Проверить количество резервуаров	Соответствует данным на сервере	QTest / Ручная проверка
3.2.3	Пустой список резервуаров	Корректно обрабатывается и отображается	Mock-сервер / QTest

3.3. Команда get_tank_params

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
3.3.1	Отправить get_tank_params для существующего резервуара	Получены volume, temp, weight, density	Mock-сервер / QTest
3.3.2	Отправить get_tank_params для	Ответ с ошибкой (error)	Mock-сервер / QTest

	несуществующего резервуара		
3.3.3	Проверить корректность данных	Все числовые значения в допустимых диапазонах	QTest / Ручная проверка

3.4. Команда get_tank_history

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
3.4.1	Отправить get_tank_history с count=100	Получены 100 записей истории	Mock-сервер / Postman
3.4.2	Проверить формат истории	Поля: ts, level, temp, volume, weight, density	QTest / JSON Schema
3.4.3	Получить историю для несуществующего резервуара	Ответ с ошибкой	Mock-сервер / QTest
3.4.4	Проверить сортировку записей	Записи отсортированы по времени (новые сверху)	QTest / Ручная проверка

3.5. Обработка ошибок

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
3.5.1	Получить ответ с полем error	Сообщение об ошибке отображается	Mock-сервер / QTest
3.5.2	Проверить код ошибки	code соответствует типу ошибки	Mock-сервер / QTest
3.5.3	Проверить текст ошибки	text содержит понятное описание	Mock-сервер / Ручная проверка

Тестирование DataManager (Менеджер данных)

4.1. Кэширование данных

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
4.1.1	После получения данных они сохраняются в кэш	Данные доступны через <code>getTank()</code>	Юнит-тесты (JUnit / Google Test)
4.1.2	Быстрый доступ к данным из кэша	Данные возвращаются без задержки	Бенчмарки / JUnit
4.1.3	Обновление данных в кэше	Новые данные перезаписывают старые	Юнит-тесты (JUnit / Google Test)
4.1.4	Проверить структуру <code>TankData</code>	Все поля присутствуют: <code>name</code> , <code>type</code> , <code>volume_max</code> , <code>volume</code> , <code>temp</code> , <code>weight</code> , <code>density</code>	JUnit / Google Test
4.1.5	Проверить структуру <code>ServerInfo</code>	Все поля присутствуют: <code>connected</code> , <code>connected_time</code> , <code>end_point</code> , <code>launch_time</code>	JUnit / Google Test

4.2. Таймер опроса (Polling)

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
4.2.1	Запустить опрос с интервалом 1000ms	Запросы отправляются каждую секунду	JUnit / JSignalSpy / Mock-сервер
4.2.2	Остановить опрос	Запросы перестают отправляться	JUnit / JSignalSpy
4.2.3	Изменить интервал опроса	Интервал меняется корректно	JUnit / JSignalSpy

4.2.4	Опрос при отключенном сервере	Ошибки обрабатываются, опрос продолжается	QTest / Симуляция отключения
-------	-------------------------------	---	------------------------------

4.3. Сигналы/Слоты

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
4.3.1	При обновлении данных срабатывает сигнал tanksUpdated()	UI получает уведомление	QTest / QSignalSpy
4.3.2	При обновлении сервера срабатывает serverInfoUpdated()	UI обновляет статус	QTest / QSignalSpy
4.3.3	При получении истории срабатывает historyReceived()	История отображается	QTest / QSignalSpy

Тестирование MainWindow (Главное окно)

5.1. Отображение главного окна

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
5.1.1	Запуск приложения	Главное окно открывается без ошибок	Ручное тестирование / QTest
5.1.2	Проверить layout окна	Все элементы расположены корректно	Визуальная проверка / QTest
5.1.3	Изменение размера окна	Элементы адаптируются к размеру	Ручное тестирование / QTest

5.2. Панель подключения

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
5.2.1	Ввести IP:port и нажать "Подключиться"	Клиент подключается к серверу	Ручное тестирование / QTest
5.2.2	Ввести невалидный IP	Ошибка, поле подсвечивается	QTest / Ручное тестирование

5.2.3	Ввести невалидный порт	Ошибка, поле подсвечивается	QTest / Ручное тестирование
5.2.4	Пустое поле	Кнопка неактивна или сообщение об ошибке	QTest / Ручное тестирование
5.2.5	Нажать "Отключиться"	Соединение разрывается	Ручное тестирование / QTest

5.3. Панель статуса

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
5.3.1	Статус подключения к серверу	Отображается "Подключен" / "Отключен"	Ручная проверка / QTest
5.3.2	Статус подключения к ПЛК	Отображается "Подключен" / "Отключен"	Ручная проверка / QTest
5.3.3	Время подключения к ПЛК	Отображается корректное время	Ручная проверка / QTest
5.3.4	Время запуска сервера	Отображается корректное время	Ручная проверка / QTest
5.3.5	Цветовая индикация	Зеленый - подключен, Красный - отключен	Визуальная проверка / QTest

5.4. Настройка интервала обновления

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
5.4.1	Выбрать интервал 1с	Запросы каждую секунду	QTest / QSignalSpy / Mock-сервер
5.4.2	Выбрать интервал 2с	Запросы каждые 2 секунды	QTest / QSignalSpy
5.4.3	Выбрать интервал 5с	Запросы каждые 5 секунд	QTest / QSignalSpy
5.4.4	Выбрать интервал 10с	Запросы каждые 10 секунд	QTest / QSignalSpy
5.4.5	Изменить интервал во время работы	Интервал меняется без перезапуска	Ручное тестирование / QTest

5.5. Таймер обновления

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
5.5.1	Таймер запускается после подключения	Данные обновляются автоматически	QTest / QSignalSpy
5.5.2	Таймер останавливается при отключении	Запросы прекращаются	QTest / QSignalSpy
5.5.3	Точность таймера	Интервал соответствует заданному	QTest / QElapsedTimer

5.6. Область резервуаров

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
5.6.1	При получении списка создаются карточки	Для каждого резервуара создана карточка	QTest / Ручная проверка
5.6.2	Динамическое добавление нового резервуара	Карточка появляется автоматически	QTest / Mock-сервер
5.6.3	Удаление резервуара	Карточка исчезает автоматически	QTest / Mock-сервер
5.6.4	Прокрутка области	Все карточки доступны при прокрутке	Ручное тестирование
5.6.5	Обновление данных в карточках	Данные обновляются без пересоздания	QTest / QSignalSpy

Тестирование TankCardWidget (Карточка резервуара)

6.1. Отображение данных

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
6.1.1	Отображается имя резервуара	Имя соответствует данным	QTest / Ручная проверка
6.1.2	Отображается тип резервуара	"Вертикальный" или "Горизонтальный"	QTest / Ручная проверка
6.1.3	Отображается уровень	Значение в сантиметрах	QTest / Ручная проверка
6.1.4	Отображается температура	Значение в °C	QTest / Ручная проверка
6.1.5	Отображается объем	Значение в литрах	QTest / Ручная проверка

6.2. Индикация уровня

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
6.2.1	QProgressBar отображает уровень	Значение пропорционально уровню	QTest / QProgressBar API
6.2.2	Уровень 0%	ProgressBar пустой	QTest / QProgressBar API
6.2.3	Уровень 100%	ProgressBar полный	QTest / QProgressBar API
6.2.4	Уровень 50%	ProgressBar заполнен наполовину	QTest / QProgressBar API

6.3. Цветовая индикация

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
6.3.1	Нормальный уровень (0-70%)	Зеленый цвет	Визуальная проверка / QTest
6.3.2	Предупредительный уровень (70-90%)	Желтый цвет	Визуальная проверка / QTest

6.3.3	Критический уровень (90-100%)	Красный цвет	Визуальная проверка / QTest
6.3.4	Изменение цвета при обновлении данных	Цвет меняется корректно	QTest / QSignalSpy

6.4. Метод updateDisplay()

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
6.4.1	Вызов updateDisplay с новыми данными	Все поля обновлены	QTest / Google Test
6.4.2	Вызов updateDisplay с теми же данными	Поля не меняются (оптимизация)	QTest / Бенчмарки
6.4.3	Вызов updateDisplay с некорректными данными	Ошибка не возникает, данные игнорируются	QTest / Google Test

Тестирование HistoryWindow (Окно истории)

7.1. Отображение истории

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
7.1.1	Открыть историю для резервуара	Окно открывается без ошибок	Ручное тестирование / QTest
7.1.2	Отображается таблица с записями	Колонки: время, уровень, температура, объем	QTest / Ручная проверка
7.1.3	Данные загружаются корректно	Все записи соответствуют данным на сервере	QTest / Mock-сервер
7.1.4	Отображается индикатор загрузки	Пока загружаются данные	QTest / Ручная проверка

7.2. Экспорт в CSV

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
7.2.1	Экспортировать историю в CSV	Файл создан, данные корректны	Ручная проверка / QFile
7.2.2	Проверить формат CSV	Разделители, заголовки корректны	Notepad++ / Excel
7.2.3	Экспорт пустой истории	Файл создан с заголовками	Ручная проверка / QFile

Тестирование потоков

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
8.1	UI не блокируется при сетевых операциях	Интерфейс остается отзывчивым	QTest / Ручное тестирование / QThread
8.2	Сигналы/слоты работают между потоками	Данные обновляются корректно	QTest / QSignalSpy
8.3	Таймеры работают в основном потоке	События обрабатываются своевременно	QTest / QTimer

Интеграционное тестирование

№	Тест-кейс	Ожидаемый результат	Инструмент/Метод
9.1	Полный цикл: Подключение → Получение данных → Обновление UI	Все этапы выполняются корректно	Интеграционные тесты / Mock-сервер
9.2	Работа с несколькими резервуарами	Все резервуары отображаются	Mock-сервер / QTest
9.3	Обновление данных с сервера	Новые данные отображаются	Mock-сервер / QTest
9.4	Переподключение при потере связи	Клиент восстанавливает соединение	Симуляция обрыва / Integration Test
9.5	Работа с историей	История загружается и отображается	Mock-сервер / QTest

Инструменты для тестирования

Инструмент	Назначение	Где используется
Qt Test (QTest)	Модульное тестирование Qt-приложений	Классы DataManager, HmiClient, виджеты
Google Test	Модульное тестирование C++ кода	Классы TankData, ServerInfo, HistoryRecord
QSignalSpy	Проверка сигналов/слотов Qt	DataManager, MainWindow
QTimer / QElapsedTimer	Проверка временных интервалов	Таймеры, Polling
Mock-сервер	Симуляция сервера для тестов	Протокол, DataManager
Postman	Проверка API и протоколов	Ручное тестирование протокола
Wireshark	Анализ сетевого трафика	Проверка TCP, формата кадров
QTest / QAbstractItemModelTester	Тестирование моделей данных	TableModel для истории
QTest / QWidget	Тестирование виджетов Qt	TankCardWidget, MainWindow
QTest / QApplication	GUI-тестирование	Главное окно, диалоги

Библиотеки для тестирования

Библиотека	Назначение	Где используется
Qt Test Framework	Тестирование Qt приложений	Все Qt-классы
Google Test / Google Mock	Модульное тестирование и моки	HmiClient, DataManager
Catch2	Альтернатива для unit-тестов	Можно добавить опционально
Boost.Test	Тестирование Boost-кода	При использовании Boost